AI506: Advanced Machine Learning

Project 2: Emotion Classifier

Table of Contents:

Task 1 - Data Preparation 1
- Dataset Loading and Structure 1
- Class Distribution Analysis 2
- Tokenisation 2
- Vocabulary Construction 2
- Encoding, Padding, and DataLoaders 3

Task 2 - Model 1 (MLP) 4
- Model Architecture 4
- Global Average Pooling 4
- Regularization 4
- Optimization and Loss Function 4
- Early Stopping 4
- Training Results and Visualisation 4
- Limitations of the model 5

Task 3 - Model 2 7
- Architecture 7
- Training and Hyperparameter Tuning 7
- Results & Evaluation 7

Task 4 - Model Analysis 11
- Without Weighted Loss 11
- Using Weighted Loss 12
- Inspection of Failure Cases 13
- Modifying Failure Cases to Achieve Correct Classification 14
- Modifying Success Cases to Obtain Misclassification 14

Task 5 - Fine-tuning a pre-trained language model (DistilBERT) 15
- Model Choice 15
- Data Pipeline 15
- Hyperparameter Tuning 15
- Final Fine-tuning 15
- Inference & Error Analysis 17

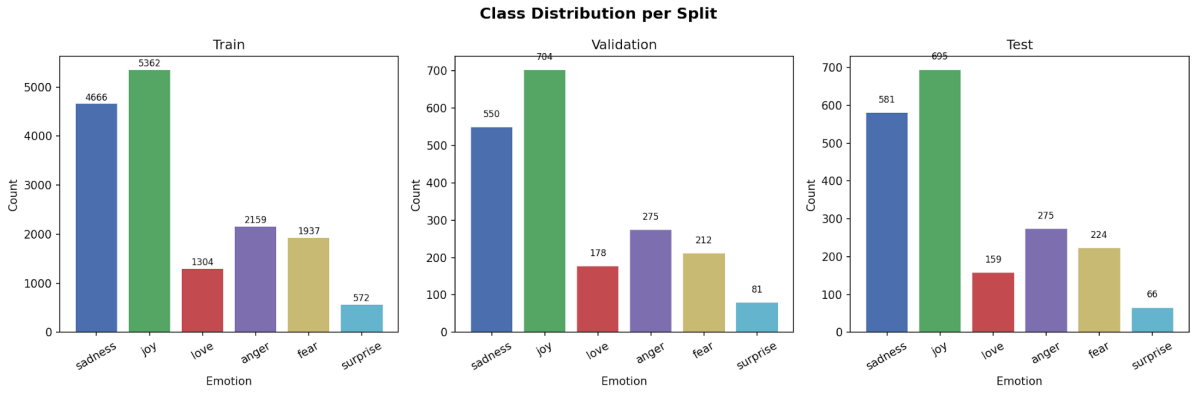
Task 1 - Data Preparation

Dataset Loading and Structure

The dair-ai/emotion dataset was loaded directly from HuggingFace using the `split` configuration, which provides pre-defined train, validation, and test splits. This avoids any manual splitting and ensures reproducibility across runs. The dataset contains short English tweets labelled across six emotion categories: sadness, joy, love, anger, fear, and surprise.

Class Distribution Analysis

The class distribution was computed and visualised for all three splits. This step is essential in any classification task because class imbalance can mislead a model into favouring majority classes. The imbalance is worth addressing during model training.



The dataset is imbalanced across all three splits, but the distribution remains consistent between train, validation, and test sets. Joy is the dominant class, accounting for roughly 33-35% of each split. Surprise is the rarest class. This imbalance can create a majority-class classifier that always predicts "joy" that achieves 33.5% accuracy, far above the 16.7% uniform random chance baseline.

This means raw accuracy alone is not a sufficient evaluation metric. Per-class metrics such as macro-averaged F1 are more informative. Other than that, a weighted loss would be more suitable to prevent the model from being biased toward the majority class.

Tokenisation

A simple regex-based tokeniser was applied, converting text to lowercase and splitting tokens based on word characters and punctuation. Lowercasing reduces vocabulary size and information loss, and keeping punctuation preserves emotional signals.

Across all splits, sequence lengths are broadly consistent. In the training set, lengths range 2-66 tokens, with a mean of 19.17, a median of 17, and a standard deviation of around 11. The 95th percentile is at 41 tokens, indicating that the vast majority of sequences are quite short.

Vocabulary Construction

The vocabulary was built exclusively from the training split to avoid any data leakage from validation or test data. Two special tokens were reserved: `<PAD>` at index 0 for sequence padding and `<UNK>` at index 1 for out-of-vocabulary words encountered at inference time.

The most frequent token is "i" with 25,859 occurrences, which is expected given that the dataset consists of first-person emotional statements. The out-of-vocabulary rate is 0% on the training set (by construction) and approximately 2.71-2.72% on the validation and test sets, which is low and indicates that the training vocabulary generalises well to unseen data.

Encoding, Padding, and DataLoaders

Each token sequence was mapped to its corresponding vocabulary index, with unknown tokens replaced by the `<UNK>` index. Sequences were then padded or truncated to a uniform maximum length of 41 tokens, chosen as the 95th percentile of training sequence lengths, covering 95% of training examples without truncation.

The resulting tensor shapes are [16000, 41] for training and [2000, 41] for both validation and test. DataLoaders were constructed with a batch size of 64, with 250 training batches and 32 batches each for validation and test. The training loder shuffles data each epoch to prevent the model from learning ordered patterns, while validation and test loaders are unshuffled to ensure consistent and comparable evaluation across epochs.

Task 2 - Model 1 (MLP)

Model Architecture

The first model implemented is a multilayer perceptron (MLP), a type of feedforward neural network. The architecture begins with an embedding layer that maps input tokens to a 128-dimensional vector space. This is followed by fully-connected hidden layers, decreasing in size from 128 → 64 → 6 neurons. We used the ReLU activation functions to introduce non-linearity into the model. The output layer is linear with 6 units, one for each emotion.

Global Average Pooling

To handle varying sentence lengths, the sequence of word embeddings is converted into a fixed-size vector by taking the mean across the sentence. This allows for an efficient way to represent the overview of a text. However, this method essentially treats the input as a “bag of words”, capturing general sentiments but losing the specific order of tokens.

Regularization

To prevent the model from just memorising the training vocabulary and overfitting, we utilised Dropout (0.5) after each hidden layer and L^2 Weight Decay (1e-4)

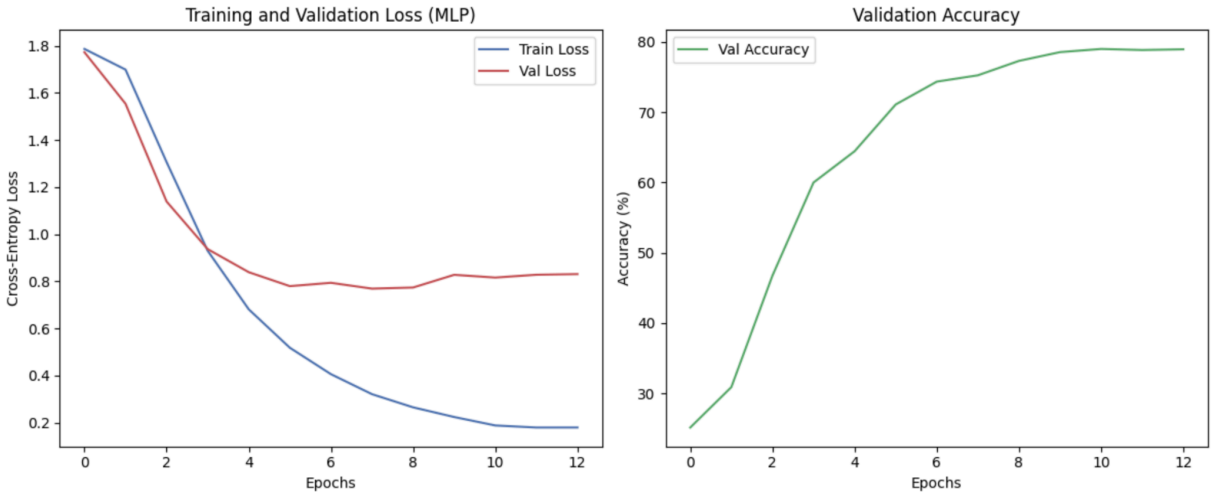
Optimization and Loss Function

For optimization, we employed AdamW paired with a learning rate scheduler (ReduceLROnPlateau). To address class imbalance within the datset, we also incorporated class weights into our cross-entropy loss function. These weights were computed using the inverse frequency of each class in the training set. This helps to appropriately penalise the model for misclassifying minority classes. Additionally, the scheduler automatically lowers the learning rate when the validation loss saturates, ensuring smoother convergence and preventing the model from overshooting the optimal global minimum. This combined approach allows us to achieve a more stable and higher final accuracy.

Early Stopping

The model was trained with an early stopping patience of 5 epochs. Training will stop if the validation loss does not decrease for 5 consecutive epochs. This allows us to capture the model at its best performance before overfitting occurs. The best model weights are saved to our output folder.

Training Results and Visualisation



```
Epoch [1] - Train Loss: 1.7869 | Val Loss: 1.7730 | Val Acc: 25.15%
Epoch [2] - Train Loss: 1.6992 | Val Loss: 1.5542 | Val Acc: 30.90%
Epoch [3] - Train Loss: 1.3064 | Val Loss: 1.1390 | Val Acc: 46.75%
Epoch [4] - Train Loss: 0.9300 | Val Loss: 0.9369 | Val Acc: 60.00%
Epoch [5] - Train Loss: 0.6813 | Val Loss: 0.8390 | Val Acc: 64.45%
Epoch [6] - Train Loss: 0.5182 | Val Loss: 0.7798 | Val Acc: 71.10%
Epoch [7] - Train Loss: 0.4060 | Val Loss: 0.7940 | Val Acc: 74.35%
Epoch [8] - Train Loss: 0.3211 | Val Loss: 0.7692 | Val Acc: 75.25%
Epoch [9] - Train Loss: 0.2652 | Val Loss: 0.7740 | Val Acc: 77.30%
Epoch [10] - Train Loss: 0.2241 | Val Loss: 0.8278 | Val Acc: 78.55%
Epoch [11] - Train Loss: 0.1883 | Val Loss: 0.8162 | Val Acc: 79.00%
Epoch [12] - Train Loss: 0.1795 | Val Loss: 0.8282 | Val Acc: 78.85%
Epoch [13] - Train Loss: 0.1796 | Val Loss: 0.8309 | Val Acc: 78.95%
Early stopping triggered! No improvement for 5 epochs.
```

From the training results, we can see that early stopping was triggered at epoch 13. While the training loss decreased steadily throughout, the validation loss began increasing from epoch 9. The divergence of the two curves indicates that the model began to overfit and memorise properties of the training set too well, rather than generalizing to unseen data. We hence reload the best weights from epoch 8, to ensure the final model used the most generalizable parameters.

```
FINAL TEST EVALUATION
Final Test Accuracy: 75.20%
Macro F1: 0.7154
              precision    recall  f1-score   support

     sadness       0.87      0.77      0.82       581
         joy       0.87      0.69      0.77       695
        love       0.45      0.86      0.59       159
       anger       0.78      0.79      0.79       275
        fear       0.74      0.75      0.75       224
    surprise       0.44      0.86      0.58        66

    accuracy                           0.75      2000
   macro avg       0.69      0.79      0.72      2000
weighted avg       0.80      0.75      0.76      2000
```

The final model is then evaluated against the test set. The final test accuracy obtained is 75.2%. This final test accuracy greatly outperforms the chance accuracy (16.7%) and majority-class accuracy (33.5%) obtained in Task 1. We can confirm that the model has successfully learnt the distinctive patterns associated with the six different emotion categories, and classify them to a certain extent.

On the other hand, the model achieves a macro F1 of 0.7154, but the weighted F1 is 0.76. This shows that per-class performance is uneven, with minority classes ("love" and "surprise") dragging the macro average down despite the weighting loss strategy.

Limitations of the model

This MLP model has two main limitations:

1. First, loss of sequential context. By using average pooling, the model treats sentences as a “bag of words”. The model recognises the words present, but not their order in the sentence. This makes it difficult for the model to understand

nuanced contexts or negation, as it fails to capture the semantic structure of language.

2. Second, performance saturation. The validation accuracy of the model saturates at around 79%. This suggests that the MLP may have reached its maximum capacity for this data representation. We hence have to transition to a more complex Transformer architecture in Task 3.

Task 3 - Model 2

Model 2 is a Transformer. What MLP couldn't achieve was understanding a sentence based on the overall words in that corresponding sentence. With the mechanism called "self-attention" in Transformer, the model can adjust the meaning of a sentence based on each word's position and contextual meaning.

Architecture

The model stacks two Transformer blocks, each containing a multi-head self-attention layer (4 heads, so each head specialises in different relationships) and a small feed-forward network (FFN) applied to each position. The model has approximately 2.2 million trainable parameters, compared to the MLP's smaller parameters. The same class-weighted cross-entropy loss from Task 2 is applied here to handle the dataset imbalance.

Training and Hyperparameter Tuning

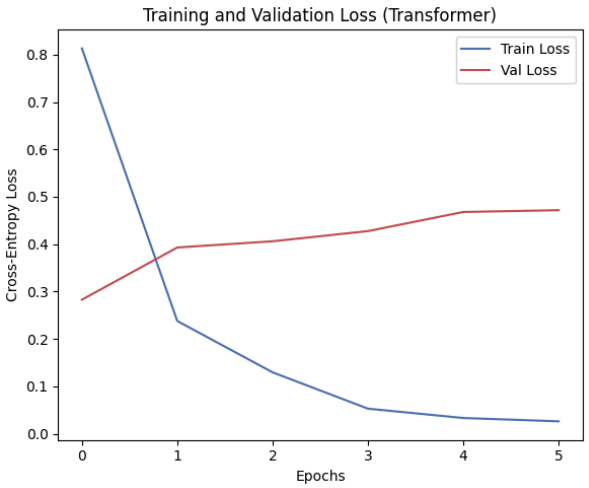
The model was trained with AdamW (lr=1e-3, weight decay=1e-4) and a ReduceLROnPlateau scheduler. Gradient clipping (max_norm=1.0) was applied at every training step. Early stopping with patience 5 triggered after just 6 epochs in both runs.

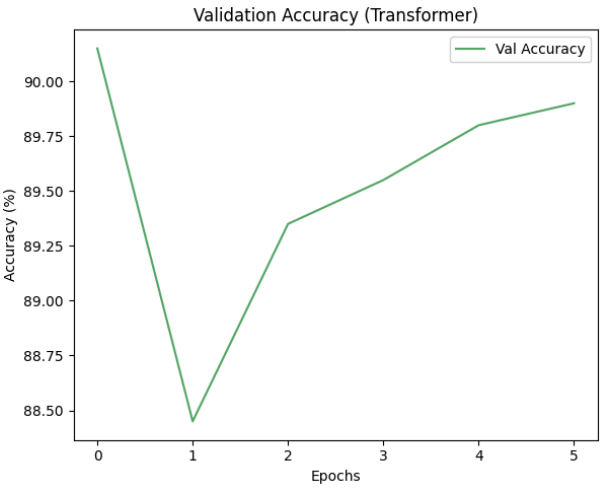
For the hyperparameter tuning, a grid search was run over 48 configurations varying dropout (0.1, 0.3, 0.4), weight decay (1e-4, 1e-3), number of layers (1, 2), FFN dimension (128, 256), and learning rate (1e-3, 5e-4). The best configuration found was dropout=0.1, weight decay=1e-4, 2 layers, FFN dim=128, lr=1e-3. This is very close to the original defaults. The FFN dimension shrunk from 256 to 128, which slightly reduced the parameter count, but all other hyperparameters remained the same.

Results & Evaluation

Before hyperparameter tuning:

```
Epoch [ 1] - Train Loss: 0.8131 | Val Loss: 0.2828 | Val Acc: 90.15%
Epoch [ 2] - Train Loss: 0.2378 | Val Loss: 0.3930 | Val Acc: 88.45%
Epoch [ 3] - Train Loss: 0.1294 | Val Loss: 0.4061 | Val Acc: 89.35%
Epoch [ 4] - Train Loss: 0.0528 | Val Loss: 0.4276 | Val Acc: 89.55%
Epoch [ 5] - Train Loss: 0.0331 | Val Loss: 0.4679 | Val Acc: 89.80%
Epoch [ 6] - Train Loss: 0.0261 | Val Loss: 0.4717 | Val Acc: 89.90%
Early stopping triggered after 6 epochs.
```



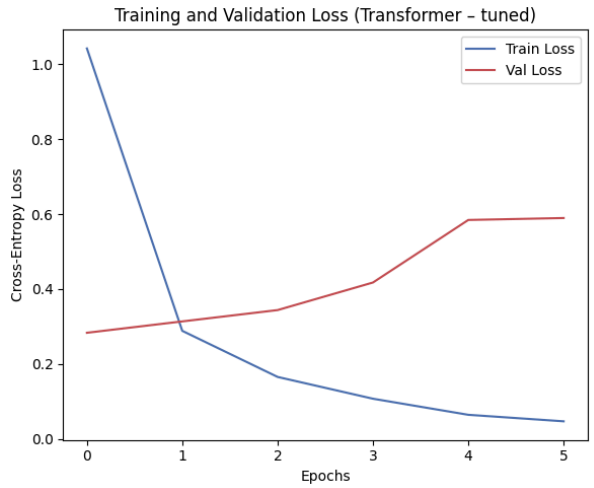


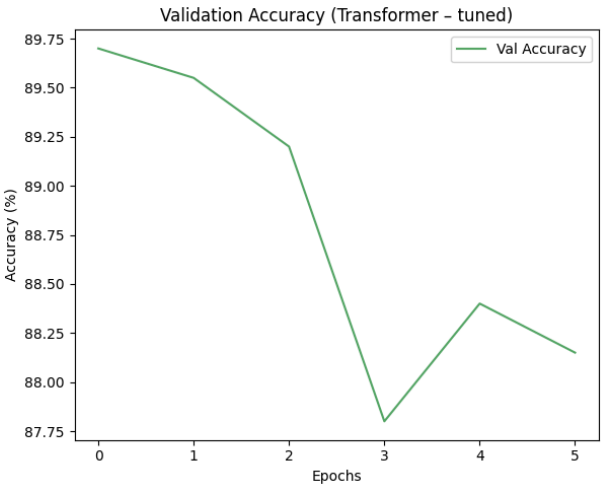

```
Final Test Accuracy : 88.15%
Macro F1            : 0.8399
```

	precision	recall	f1-score	support
sadness	0.96	0.90	0.93	581
joy	0.97	0.84	0.90	695
love	0.67	0.94	0.79	159
anger	0.86	0.94	0.90	275
fear	0.84	0.84	0.84	224
surprise	0.56	0.88	0.69	66
accuracy			0.88	2000
macro avg	0.81	0.89	0.84	2000
weighted avg	0.90	0.88	0.89	2000

After hyperparameter tuning (grid search):

```
Epoch [ 1] - Train Loss: 1.0421 | Val Loss: 0.2835 | Val Acc: 89.70%
Epoch [ 2] - Train Loss: 0.2887 | Val Loss: 0.3138 | Val Acc: 89.55%
Epoch [ 3] - Train Loss: 0.1657 | Val Loss: 0.3441 | Val Acc: 89.20%
Epoch [ 4] - Train Loss: 0.1076 | Val Loss: 0.4176 | Val Acc: 87.80%
Epoch [ 5] - Train Loss: 0.0647 | Val Loss: 0.5845 | Val Acc: 88.40%
Epoch [ 6] - Train Loss: 0.0473 | Val Loss: 0.5897 | Val Acc: 88.15%
Early stopping after epoch 6.
```






```
FINAL TEST EVALUATION (Transformer - tuned)
Best hyperparameters : {'dropout': 0.1, 'weight_decay': 0.0001, 'num_layers': 2, 'ffn_dim': 128, 'lr': 0.001, 'val_loss': 0.28351194388233125, 'val_acc': 89.7}
Final Test Accuracy : 88.55%
Macro F1 : 0.8429
```

	precision	recall	f1-score	support
sadness	0.95	0.90	0.93	581
joy	0.96	0.87	0.91	695
love	0.68	0.92	0.79	159
anger	0.86	0.92	0.89	275
fear	0.91	0.81	0.86	224
surprise	0.54	0.94	0.69	66
accuracy			0.89	2000
macro avg	0.82	0.89	0.84	2000
weighted avg	0.90	0.89	0.89	2000

Training loss drops steeply and consistently toward near-zero, while validation loss rises after epoch 1. This is the sign of overfitting. The model is memorising the training data rather than learning patterns that generalise. Despite this, validation accuracy remains high (around 88-90%) because the model learned strong representations in the first epoch.

The tuned model's validation loss rises faster and higher than the pre-tuning model (reaching ~0.59 vs ~0.47 at epoch 6), and its accuracy curve is more unstable, dipping to 87.80% at epoch 4.

However, the final test accuracy is actually slightly better after tuning: 88.55% vs 88.15%, and macro F1 improves a bit from 0.8399 to 0.8429. The reason the graph looks worse is

that the tuned model uses a warmup + cosine decay learning rate schedule instead of ReduceLROnPlateau. This schedule starts with a higher effective learning rate during warmup, which causes a noisier early training, but then becomes better by test time.

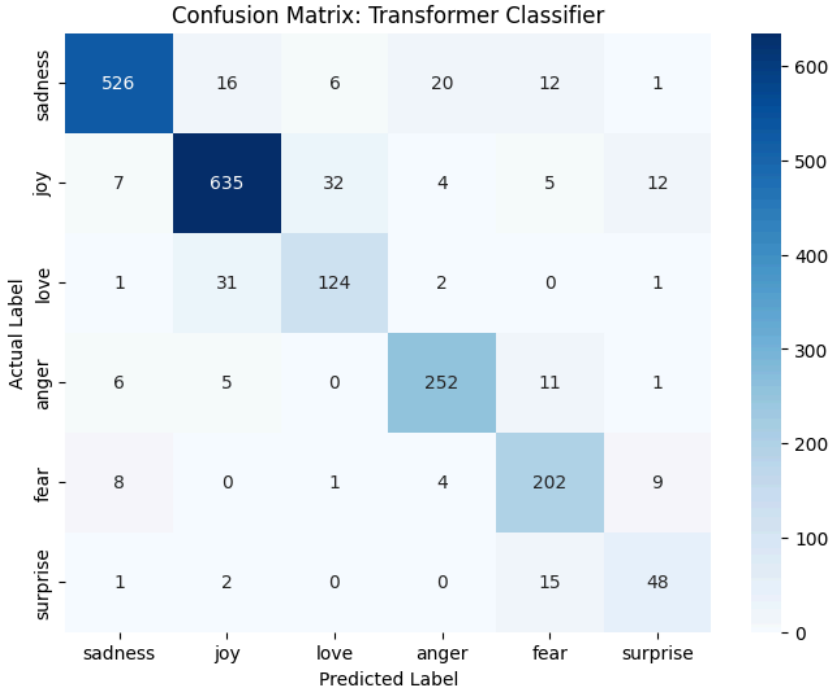
The tuned Transformer achieves 88.55% accuracy and a macro F1 of 0.8429, a substantial improvement over the MLP's 75.20% and macro F1 of 0.7154. The model still over-predicts minority classes due to the class weighting, but it does more accurately than the MLP. Overall, the Transformer handles the imbalance and the classification task a lot better than the MLP.

Task 4 - Model Analysis

We chose to perform a basic analysis for Model 2 (Transformer Classifier), as it significantly outperforms Model 1 (MLP). We felt that it would be more interesting to analyse Model 2 performance, as it is a more complex model with context awareness, instead of just word averaging.

In this project, we're working with class imbalance, and we deal with it using weighted loss. Hence, we want to know if the weighted loss actually has any impact.

Without Weighted Loss



In the confusion matrix, the diagonal squares represent correct predictions. Since the diagonal squares are darker in colour than the rest of the matrix, we can confirm that the model is mostly returning true positive cases, and has successfully learned to recognise and classify emotional patterns to a large extent.

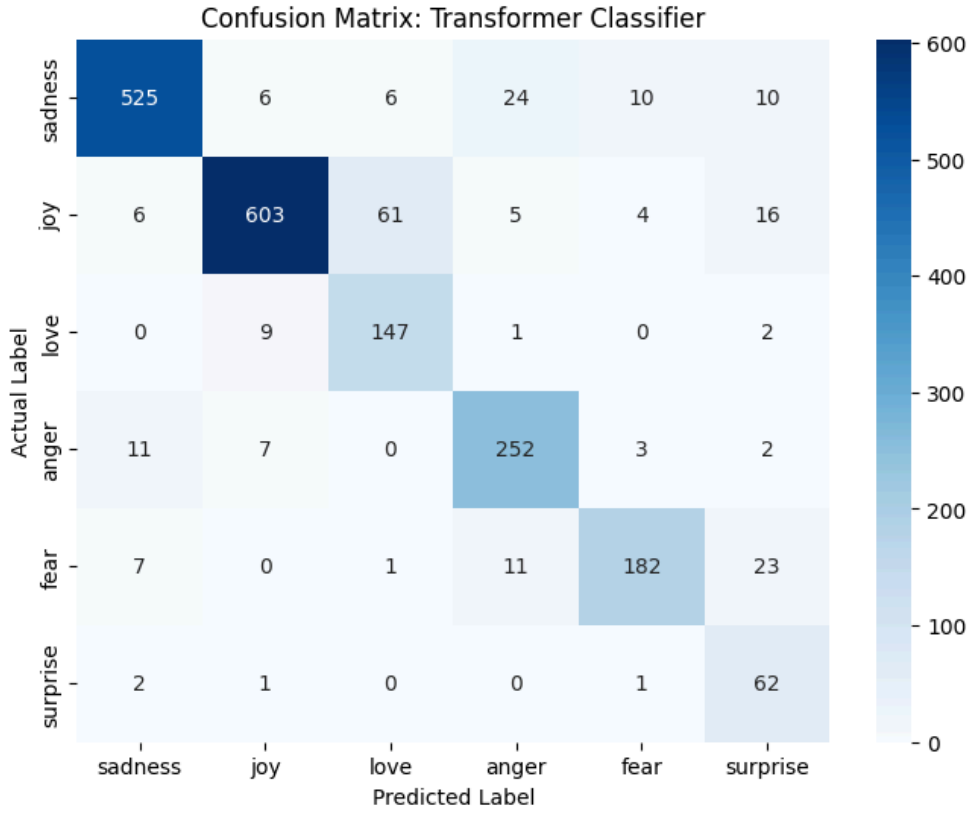
From the confusion matrix, we can also note that the model struggles to distinguish between similar positive emotions. For the actual label joy, the majority of its misclassification is from the predicted label love. The model also struggles to distinguish between similar negative emotions. For the actual label anger, the majority of its misclassification comes from the predicted label fear. This is because these emotions are semantically similar.

However, the model is good at distinguishing between positive versus negative emotions.

The model performs significantly worse on the emotion “Surprise” (72.7%), as compared to the emotion “Joy” (91.3%). This could be because of class imbalance in the training data, where there was significantly less data on “Surprise” class (3.3%) as compared to “Joy” class (35.2%). This limits the model’s ability to learn the distinct features of “Surprise” class,

leading it to often misclassify these cases as semantically similar high-arousal emotions like “Fear” or “Joy”.

Using Weighted Loss



In the updated confusion matrix using the weighted loss strategy, we can see how the accurate predictions have improved for the minority classes ("love" and "surprise"). The cost, however, is more misclassification of "joy" and "fear" classes.

This is because "love" and "joy" are hard to distinguish, which is shown in an example below.

```
Text: i explain why i clung to a relationship with a boy who was in
many ways immature and uncommitted despite the excitement i should have
been feeling for getting accepted into the masters program at the
university of virginia
  -> Actual Emotion:    joy
  -> Model's Guess:     love
```

We believe that humans can make the same error to predict love vs joy emotions.

Similarly, "fear" and "surprise" classes can overlap. "Fear" can also be misclassified as "anger" or "sadness". An example is given below.

```
Text: i was so uncomfortable and feeling weird feelings but wasn t sure
if they were contractions since i never really felt contractions with
jared until they jacked me up with pitocin
  -> Actual Emotion:    surprise
  -> Model's Guess:     fear
```

Inspection of Failure Cases

These examples are extracted before and after weighted loss was incorporated, and the sentences might be different, but semantically similar.

Example 1 (before weighted loss): "*im not sure the feeling of loss will ever go away but it may dull to a sweet feeling of nostalgia at what i shared in this life with my dad…*". Actual: Sadness | Predicted: Love.

Example 1 (after weighted loss): "*im not sure the feeling of loss will ever go away but it may dull to a sweet feeling of nostalgia at what i shared in this life with my dad…*". Actual: Sadness | Predicted: Love.

This sentence is long and complex. Although the main theme of the sentence was the "loss" of his "dad", there are many words present that are strongly associated with love, such as "sweet", "nostalgia", "shared". The model was likely distracted by the many positive sentiment words in the sentence, and failed to understand the nuanced context of loss and grief present.

Example 2 (before weighted loss): "*i feel agitated with myself that i did not foresee her frustrations earlier leading to the ending of our relationship*". Actual: Fear | Prediction: Anger.

Example 2 (after weighted loss): "*i don t feel particularly agitated*". Actual: Fear | Prediction: Anger.

Both "Fear" and "Anger" are high-arousal and negative emotions. Since the word "agitated" is a strong trigger for anger in many contexts, the model struggled to differentiate the inward-directed anxiety (fear) from the outward-directed frustration (anger).

Example 3 (before weighted loss): "*i feel very honoured to be included in a magazine which prioritises health and clean living so highly im curious do any of you…*". Actual: Joy | Prediction: Surprise.

Example 3 (after weighted loss): "*i feel very honoured to be included in a magazine which prioritises health and clean living so highly im curious do any of you…*". Actual: Joy | Prediction: Surprise.

It seems like the word "curious" is a strong indicative word for "Surprise" in the training data. The model prioritised the inquisitive nature of the second half of the sentence, instead of the joyful feeling at the start.

To summarise, we observe the following possible reasons for failure.

1. First, lexical dominance of certain words. The model often prioritises individual high-weight tokens (eg. "sweet", "curious") over the overall semantic structure of the sentence.
2. Second, confusion in the context of similar emotions. The model struggles to differentiate between emotions that share similar intensities and negative connotations.
3. Third, complex long sentences. The model struggles to identify which part of the sentence holds the "primary" emotion.
4. Fourth, hidden emotions. The model has difficulties reading behind the lines like humans do because it relies on the lexicon representations.

Modifying Failure Cases to Achieve Correct Classification

Example	Modification
Example 1: "*im not sure the feeling of loss will ever go away but it may dull to a sweet feeling of nostalgia at what i shared in this life with my dad…*". Actual: Sadness | Predicted: Love.	Changed "sweet" to "painful" → model successfully predicts "Sadness". This proves that the model was highly sensitive to adjectives and over-weighed the importance of the word "sweet".
Example 2: "*i feel agitated with myself that i did not foresee her frustrations earlier leading to the ending of our relationship*". Actual: Fear | Prediction: Anger.	Changed "agitated" to "anxious" → model successfully predicts "Fear" The model relies on the arousal-level of the word. "Agitated" usually hints towards "Anger", and "Anxious" hints towards "Fear".
Example 3: "*i feel very honoured to be included in a magazine which prioritises health and clean living so highly im curious do any of you…*". Actual: Joy | Prediction: Surprise.	Changed "curious" to "wondering" → model successfully predicts "Joy". By removing "curious" which was a strong indicative word for "Joy", the model was able to better prioritise other parts of the sentence such as "honoured".

```
Text: im not sure the feeling of loss will ever go away but it may dull to a painful feeling of nostalgia at what i shared in this life with my dad…
Predicted Label: sadness
Actual Label: sadness
```

```
Text: i feel anxious with myself that i did not foresee her frustrations earlier  leading to the ending of our relationship
Predicted Label: fear
Actual Label: fear
```

```
Text: i feel very honoured to be included in a magazine which prioritises health and clean living so highly im wondering do any of you…
Predicted Label: joy
Actual Label: joy
```

Modifying Success Cases to Obtain Misclassification

Example	Modification
Example 1: "*i cant walk into a shop anywhere where i do not feel uncomfortable*". Actual: Fear | Predicted: Fear.	Changed "uncomfortable" to "safe" → model misclassifies example as "Joy" The model focuses too much on the adjective present in the sentence, and fails to see the overall logical representation of the sentence, including the negation "not" present. "Uncomfortable" also seems to be a major trigger word for "Fear".

```
Text: i cant walk into a shop anywhere where i do not feel safe
Predicted Label: joy
Actual Label: fear
```

Task 5 - Fine-tuning a pre-trained language model (DistilBERT)

Model Choice

distilbert-base-uncased was selected because of some reasons as follow:

- Being smaller and faster than bert-base-uncased (66M vs 110M parameters), but still retains BERT's capabilities
- Even though "small", it is better than the custom Transformer (Model 2) from Task 3
- Well-suited to short texts (average ~19 tokens)

Data Pipeline

Because DistilBERT uses its own WordPiece tokeniser, the custom vocabulary from Task 1 was not reused. Raw texts were re-tokenised with DistilBertTokenizerFast with a result of maximum length 64, which covers most of the dataset. A custom PyTorch Dataset class (EmotionDataset) wraps the tokenised encodings and label tensors, exposing input_ids, attention_mask, and labels per sample.

Hyperparameter Tuning

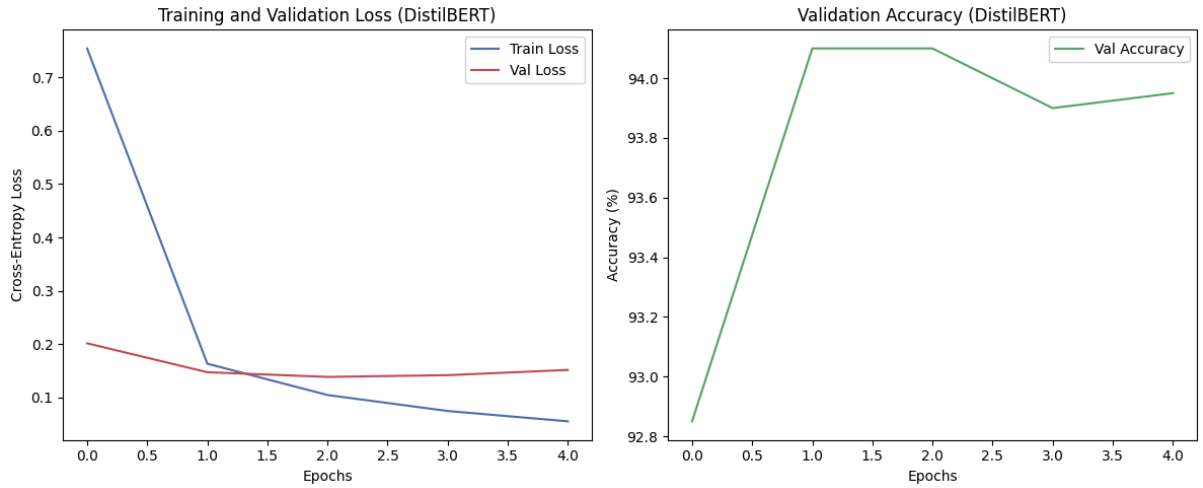
Fine-tuning BERT-family models is particularly sensitive to learning rate (too high causes catastrophic forgetting of pre-trained weights) and classifier dropout. Thus, a standard grid search over 9 configurations was tested over three learning rates (2e-5, 3e-5, 5e-5) and three dropout values (0.1, 0.2, 0.3).

It turned out that dropout had almost no effect, probably because the pre-trained weights are already robust and the fine-tuning is very short (3 epochs). The learning rate mattered more, with lr=3e-5 achieving the best validation loss of 0.1345, making it the chosen configuration.

Final Fine-tuning

A fresh DistilBERT model was fine-tuned with the best hyperparameters for up to 5 epochs (early-stop patience=2).

```
Epoch [ 1] - Train Loss: 0.7541 | Val Loss: 0.2017 | Val Acc: 92.85%
Epoch [ 2] - Train Loss: 0.1635 | Val Loss: 0.1475 | Val Acc: 94.10%
Epoch [ 3] - Train Loss: 0.1048 | Val Loss: 0.1387 | Val Acc: 94.10%
Epoch [ 4] - Train Loss: 0.0749 | Val Loss: 0.1420 | Val Acc: 93.90%
Epoch [ 5] - Train Loss: 0.0555 | Val Loss: 0.1518 | Val Acc: 93.95%
Early stopping after epoch 5.
```




```
FINAL TEST EVALUATION (DistilBERT fine-tuned)
Final Test Accuracy: 93.05%
Macro F1: 0.8831
              precision    recall  f1-score   support

     sadness       0.97      0.96      0.97       581
         joy       0.95      0.95      0.95       695
        love       0.82      0.87      0.84       159
       anger       0.90      0.95      0.92       275
        fear       0.89      0.93      0.91       224
    surprise       0.93      0.58      0.71        66

    accuracy                           0.93      2000
   macro avg       0.91      0.87      0.88      2000
weighted avg       0.93      0.93      0.93      2000
```

Unlike the custom Transformer which showed severe overfitting, DistilBERT's training is much better. Validation loss actually improves from epoch 1 (0.2017) to epoch 3 (0.1387), and only begins to increase slowly after that. This is because the pre-trained weights provide an excellent starting point, and the model doesn't need many updates and doesn't memorise the training data as aggressively.

Comparing across the three tasks: the MLP scored 75.20% (macro F1: 0.72), the tuned Transformer scored 88.55% (macro F1: 0.84), and the final DistilBERT reached 93.05% (macro F1: 0.88). The one remaining weak spot is surprise, with an F1 of only 0.71, but its precision is actually very high (0.93) but recall drops to 0.58, meaning the model misses many true surprise examples. The class-weighted loss was used, but less dominant here because DistilBERT's pre-trained representations are strong enough that it doesn't need to overcorrect as aggressively as the MLP and Transformer.

Inference & Error Analysis

A predict_bert() helper was implemented to classify arbitrary text strings. We collected the first 10 misclassified examples to understand failures.

Some categories of test inputs were explored:

- Clear positives / negatives: the model correctly classified with high confidence
- Ambiguous inputs (e.g., racing heart): model captured the ambiguity between fear and surprise
- Minimal word substitution: swapping 'loved' for 'watched' shifted the confidence from 100% love to a bit of sadness, showing the model's sensitivity to semantics
- Incremental deletion: progressively removing emotional phrases (e.g., 'kissing', 'love', 'sweet lips') caused gradual prediction confidence, demonstrating that the model relies on particular word cues

This shows the model is heavily lexically driven. It detects specific emotional keywords, but it is sensitive to single word changes.